TikTok - REST API Design and Implementation

Overview

This worksheet guides you through creating a RESTful API for your TikTok. We'll focus on designing and implementing backend endpoints that will communicate with your Next.js frontend.

API Design Overview

Resources:
Videos
Users
Comments

Endpoints	GET	POST	PUT	DELETE
/api/videos	Get all videos	Create new video	-	-
/api/videos/:id	Get video by ID	Update video	-	Delete video
/api/videos/:id/comments	Get video comments	-	-	-
/api/videos/:id/likes	Get video likes	Like video	-	Unlike video
/api/users	Get all users	Create user	-	-
/api/users/:id	Get user by ID	-	Update user	Delete user
/api/users/:id/videos	Get user videos	-	-	-
/api/users/:id/followers	Get followers	Follow user	-	Unfollow user
/api/users/:id/following	Get following users	-	-	-
/api/comments	Get all comments	Create comment	-	-
/api/comments/:id	Get comment by ID	-	Update comment	Delete comment
/api/comments/:id/likes	Get comment likes	Like comment	-	Unlike comment

Part 1: Setting Up Your Express Backend

Step 1: Initialize your project

Create a new directory for your project and initialize it:

```
mkdir -p server

cd server

npm init -y
```

Step 2: Install required dependencies

```
npm install express cors morgan body-parser dotenv

npm install --save-dev nodemon
```

Dependencies details:

- `express`: Web server framework
- `cors`: Enable Cross-Origin Resource Sharing
- `morgan`: HTTP request logger
- `body-parser`: Parse incoming request bodies
- `dotenv`: Load environment variables from .env file

Step 3: Create project structure

```
mkdir -p src/controllers src/routes src/models src/middleware src/utils

touch src/app.js src/server.js .env
```

Step 4: Configure your environment variables

In `.env`:

Step 5: Set up the Express application

In `src/app.js`:

```
const express = require('express');
const cors = require('cors');
const morgan = require('morgan');
const bodyParser = require('body-parser');

// Import routes
const videoRoutes = require('./routes/videos');
const userRoutes = require('./routes/users');
const commentRoutes = require('./routes/comments');

// Initialize Express app
const app = express();

// Middleware
app.use(morgan('dev')); // Logging
app.use(cors()); // Enable CORS for all routes
app.use(bodyParser.json()); // Parse JSON bodies
app.use(bodyParser.urlencoded({ extended: true })); // Parse URL-encoded bodies

app.use((req, res, next) => {
  // Check if the client accepts JSON
  if (!req.accepts('application/json')) {
    return res.status(406).json({
      error: 'Not Acceptable',
      message: 'This API only supports application/json'
    });
  }

  // Set Content-Type header for responses
  res.setHeader('Content-Type', 'application/json');

  next();
});

// API Routes
app.use('/api/videos', videoRoutes);
app.use('/api/users', userRoutes);
app.use('/api/comments', commentRoutes);

// Root route
app.get('/', (req, res) => {
  res.json({ message: 'Welcome to TikTok API' });
});

// 404 handler
app.use((req, res, next) => {
  res.status(404).json({ error: 'Not Found' });
});

// Error handler
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({
    error: 'Internal Server Error',
    message: process.env.NODE_ENV === 'development' ? err.message : undefined
  });
});

module.exports = app;
```

Step 6: Create the server entry point

In `src/index.js`:

```
require('dotenv').config();
const app = require('./app');

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`Server running on port http://localhost:${PORT} in ${process.env.NODE_ENV} mode`);
});
```

Step 7: Update package.json with scripts

Update the `scripts` section in `package.json`:

```
  "scripts": {
    "start": "node src/index.js",
    "dev": "nodemon src/index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
```

Part 2: RESTful API Implementation

Step 1: Create Data Models (In-Memory for now)

Since we're not using a database yet, let's create in-memory data structures:

In `src/models/index.js`. Copy the data from [HERE]() to your `src/models/index.js`.

Step 2: Implement Controllers

Let's start with the video controller:

In `src/controllers/videoController.js`:

```
const dataStore = require('../models');

// GET all videos
const getAllVideos = (req, res) => {
  res.status(200).json(dataStore.videos);
};

// GET video by ID
const getVideoById = (req, res) => {
  const videoId = parseInt(req.params.id);
  const video = dataStore.videos.find(v => v.id === videoId);

  if (!video) {
    return res.status(404).json({ error: 'Video not found' });
  }

  res.status(200).json(video);
};

// POST create a new video
const createVideo = (req, res) => {
  const { title, description, url, userId } = req.body;

  // Basic validation
  if (!title || !url || !userId) {
    return res.status(400).json({ error: 'Required fields missing' });
  }

  // Check if user exists
  const userExists = dataStore.users.some(user => user.id === parseInt(userId));
  if (!userExists) {
    return res.status(400).json({ error: 'User does not exist' });
  }

  const newVideo = {
    id: dataStore.nextIds.videos++,
    title,
    description: description || '',
    url,
    userId: parseInt(userId),
    likes: [],
    createdAt: new Date().toISOString()
  };

  dataStore.videos.push(newVideo);

  res.status(201).json(newVideo);
};
```

```
50  // PUT update a video
51  const updateVideo = (req, res) => {
52    const videoId = parseInt(req.params.id);
53    const videoIndex = dataStore.videos.findIndex(v => v.id === videoId);
54
55    if (videoIndex === -1) {
56      return res.status(404).json({ error: 'Video not found' });
57    }
58
59    const { title, description, url } = req.body;
60    const video = dataStore.videos[videoIndex];
61
62    // Update fields if provided
63    if (title) video.title = title;
64    if (description !== undefined) video.description = description;
65    if (url) video.url = url;
66
67    video.updatedAt = new Date().toISOString();
68
69    res.status(200).json(video);
70  };
71
72  // DELETE a video
73  const deleteVideo = (req, res) => {
74    const videoId = parseInt(req.params.id);
75    const videoIndex = dataStore.videos.findIndex(v => v.id === videoId);
76
77    if (videoIndex === -1) {
78      return res.status(404).json({ error: 'Video not found' });
79    }
80
81    // Remove the video
82    dataStore.videos.splice(videoIndex, 1);
83
84    // Also remove associated comments
85    dataStore.comments = dataStore.comments.filter(c => c.videoId !== videoId);
86
87    res.status(204).end();
88  };
89
90  // GET video comments
91  const getVideoComments = (req, res) => {
92    const videoId = parseInt(req.params.id);
93    const video = dataStore.videos.find(v => v.id === videoId);
94
95    if (!video) {
96      return res.status(404).json({ error: 'Video not found' });
97    }
98
99    const comments = dataStore.comments.filter(c => c.videoId === videoId);
100   res.status(200).json(comments);
101 };
```

```
103   // GET video likes
104   const getVideoLikes = (req, res) => {
105     const videoId = parseInt(req.params.id);
106     const video = dataStore.videos.find(v => v.id === videoId);
107
108     if (!video) {
109       return res.status(404).json({ error: 'Video not found' });
110     }
111
112     const likedUsers = video.likes.map(userId => {
113       return dataStore.users.find(user => user.id === userId);
114     }).filter(Boolean);
115
116     res.status(200).json(likedUsers);
117   };
118
119   // POST like a video
120   const likeVideo = (req, res) => {
121     const videoId = parseInt(req.params.id);
122     const { userId } = req.body;
123
124     if (!userId) {
125       return res.status(400).json({ error: 'userId is required' });
126     }
127
128     const userIdInt = parseInt(userId);
129     const video = dataStore.videos.find(v => v.id === videoId);
130     const user = dataStore.users.find(u => u.id === userIdInt);
131
132     if (!video) {
133       return res.status(404).json({ error: 'Video not found' });
134     }
135
136     if (!user) {
137       return res.status(404).json({ error: 'User not found' });
138     }
139
140     // Check if already liked
141     if (video.likes.includes(userIdInt)) {
142       return res.status(409).json({ error: 'User already liked this video' });
143     }
144
145     // Add like
146     video.likes.push(userIdInt);
147
148     res.status(201).json({ message: 'Video liked successfully' });
149   };
```

```
151  // DELETE unlike a video
152  const unlikeVideo = (req, res) => {
153    const videoId = parseInt(req.params.id);
154    const { userId } = req.body;
155
156    if (!userId) {
157      return res.status(400).json({ error: 'userId is required' });
158    }
159
160    const userIdInt = parseInt(userId);
161    const video = dataStore.videos.find(v => v.id === videoId);
162
163    if (!video) {
164      return res.status(404).json({ error: 'Video not found' });
165    }
166
167    // Check if user has liked the video
168    const likeIndex = video.likes.indexOf(userIdInt);
169    if (likeIndex === -1) {
170      return res.status(404).json({ error: 'Like not found' });
171    }
172
173    // Remove like
174    video.likes.splice(likeIndex, 1);
175
176    res.status(204).end();
177  };
178
179  module.exports = {
180    getAllVideos,
181    getVideoById,
182    createVideo,
183    updateVideo,
184    deleteVideo,
185    getVideoComments,
186    getVideoLikes,
187    likeVideo,
188    unlikeVideo
189  };
```

Now let's implement the user controller:

In `src/controllers/userController.js`:

```
const dataStore = require('../models');

// GET all users
const getAllUsers = (req, res) => {
  res.status(200).json(dataStore.users);
};

// GET user by ID
const getUserById = (req, res) => {
  const userId = parseInt(req.params.id);
  const user = dataStore.users.find(u => u.id === userId);

  if (!user) {
    return res.status(404).json({ error: 'User not found' });
  }

  res.status(200).json(user);
};

// POST create a new user
const createUser = (req, res) => {
  const { username, email, name } = req.body;

  // Basic validation
  if (!username || !email) {
    return res.status(400).json({ error: 'Required fields missing' });
  }

  // Check if username or email already exists
  const usernameExists = dataStore.users.some(user => user.username === username);
  const emailExists = dataStore.users.some(user => user.email === email);

  if (usernameExists) {
    return res.status(409).json({ error: 'Username already taken' });
  }

  if (emailExists) {
    return res.status(409).json({ error: 'Email already registered' });
  }

  const newUser = {
    id: dataStore.nextIds.users++,
    username,
    email,
    name: name || username,
    followers: [],
    following: [],
    createdAt: new Date().toISOString()
  };
```

```
  dataStore.users.push(newUser);

  res.status(201).json(newUser);
};

// PUT update a user
const updateUser = (req, res) => {
  const userId = parseInt(req.params.id);
  const userIndex = dataStore.users.findIndex(u => u.id === userId);

  if (userIndex === -1) {
    return res.status(404).json({ error: 'User not found' });
  }

  const { name, email } = req.body;
  const user = dataStore.users[userIndex];

  // Update fields if provided
  if (name) user.name = name;
  if (email) {
    // Check if email is already used by another user
    const emailExists = dataStore.users.some(u => u.email === email && u.id !== userId);
    if (emailExists) {
      return res.status(409).json({ error: 'Email already registered' });
    }
    user.email = email;
  }

  user.updatedAt = new Date().toISOString();

  res.status(200).json(user);
};

// DELETE a user
const deleteUser = (req, res) => {
  const userId = parseInt(req.params.id);
  const userIndex = dataStore.users.findIndex(u => u.id === userId);

  if (userIndex === -1) {
    return res.status(404).json({ error: 'User not found' });
  }

  // Remove the user
  dataStore.users.splice(userIndex, 1);

  // Also remove user's videos and comments
  dataStore.videos = dataStore.videos.filter(v => v.userId !== userId);
  dataStore.comments = dataStore.comments.filter(c => c.userId !== userId);
```

```
  // Remove user from followers/following lists
  dataStore.users.forEach(user => {
    user.followers = user.followers.filter(id => id !== userId);
    user.following = user.following.filter(id => id !== userId);
  });

  res.status(204).end();
};

// GET user videos
const getUserVideos = (req, res) => {
  const userId = parseInt(req.params.id);
  const user = dataStore.users.find(u => u.id === userId);

  if (!user) {
    return res.status(404).json({ error: 'User not found' });
  }

  const videos = dataStore.videos.filter(v => v.userId === userId);
  res.status(200).json(videos);
};

// GET user followers
const getUserFollowers = (req, res) => {
  const userId = parseInt(req.params.id);
  const user = dataStore.users.find(u => u.id === userId);

  if (!user) {
    return res.status(404).json({ error: 'User not found' });
  }

  const followers = user.followers.map(followerId => {
    return dataStore.users.find(u => u.id === followerId);
  }).filter(Boolean);

  res.status(200).json(followers);
};

// POST follow user
const followUser = (req, res) => {
  const userToFollowId = parseInt(req.params.id);
  const { followerId } = req.body;

  if (!followerId) {
    return res.status(400).json({ error: 'followerId is required' });
  }
```

```
146
147     const followerIdInt = parseInt(followerId);
148     const userToFollow = dataStore.users.find(u => u.id === userToFollowId);
149     const follower = dataStore.users.find(u => u.id === followerIdInt);
150
151     if (!userToFollow) {
152       return res.status(404).json({ error: 'User to follow not found' });
153     }
154
155     if (!follower) {
156       return res.status(404).json({ error: 'Follower not found' });
157     }
158
159     // Can't follow yourself
160     if (userToFollowId === followerIdInt) {
161       return res.status(400).json({ error: 'Users cannot follow themselves' });
162     }
163
164     // Check if already following
165     if (userToFollow.followers.includes(followerIdInt)) {
166       return res.status(409).json({ error: 'Already following this user' });
167     }
168
169     // Add follower relationship
170     userToFollow.followers.push(followerIdInt);
171     follower.following.push(userToFollowId);
172
173     res.status(201).json({ message: 'User followed successfully' });
174   };
175
176   // DELETE unfollow user
177   const unfollowUser = (req, res) => {
178     const userToUnfollowId = parseInt(req.params.id);
179     const { followerId } = req.body;
180
181     if (!followerId) {
182       return res.status(400).json({ error: 'followerId is required' });
183     }
184
185     const followerIdInt = parseInt(followerId);
186     const userToUnfollow = dataStore.users.find(u => u.id === userToUnfollowId);
187     const follower = dataStore.users.find(u => u.id === followerIdInt);
188
189     if (!userToUnfollow || !follower) {
190       return res.status(404).json({ error: 'User not found' });
191     }
192
193     // Check if following relationship exists
194     const followerIndex = userToUnfollow.followers.indexOf(followerIdInt);
195     const followingIndex = follower.following.indexOf(userToUnfollowId);
```

```
  if (followerIndex === -1 || followingIndex === -1) {
    return res.status(404).json({ error: 'Follow relationship not found' });
  }

  // Remove follower relationship
  userToUnfollow.followers.splice(followerIndex, 1);
  follower.following.splice(followingIndex, 1);

  res.status(204).end();
};

module.exports = {
  getAllUsers,
  getUserById,
  createUser,
  updateUser,
  deleteUser,
  getUserVideos,
  getUserFollowers,
  followUser,
  unfollowUser
};
```

Task: And finally, implement similarly for the comment controller in `src/controllers/commentController.js`.

Step 4: Implement Routes

Let's create the route files:

In `src/routes/videos.js`:

```
const express = require('express');
const router = express.Router();
const videoController = require('../controllers/videoController');

// GET /api/videos - Get all videos
router.get('/', videoController.getAllVideos);

// POST /api/videos - Create new video
router.post('/', videoController.createVideo);

// GET /api/videos/:id - Get video by ID
router.get('/:id', videoController.getVideoById);

// PUT /api/videos/:id - Update video
router.put('/:id', videoController.updateVideo);

// DELETE /api/videos/:id - Delete video
router.delete('/:id', videoController.deleteVideo);

// GET /api/videos/:id/comments - Get video comments
router.get('/:id/comments', videoController.getVideoComments);

// GET /api/videos/:id/likes - Get video likes
router.get('/:id/likes', videoController.getVideoLikes);

// POST /api/videos/:id/likes - Like video
router.post('/:id/likes', videoController.likeVideo);

// DELETE /api/videos/:id/likes - Unlike video
router.delete('/:id/likes', videoController.unlikeVideo);

module.exports = router;
```

In `src/routes/users.js`:

```
const express = require('express');
const router = express.Router();
const userController = require('../controllers/userController');

// GET /api/users - Get all users
router.get('/', userController.getAllUsers);

// POST /api/users - Create new user
router.post('/', userController.createUser);

// GET /api/users/:id - Get user by ID
router.get('/:id', userController.getUserById);

// PUT /api/users/:id - Update user
router.put('/:id', userController.updateUser);

// DELETE /api/users/:id - Delete user
router.delete('/:id', userController.deleteUser);

// GET /api/users/:id/videos - Get user videos
router.get('/:id/videos', userController.getUserVideos);

// GET /api/users/:id/followers - Get followers
router.get('/:id/followers', userController.getUserFollowers);

// POST /api/users/:id/followers - Follow user
router.post('/:id/followers', userController.followUser);

// DELETE /api/users/:id/followers - Unfollow user
router.delete('/:id/followers', userController.unfollowUser);

module.exports = router;
```

In `src/routes/comments.js`:

```
const express = require('express');
const router = express.Router();
const commentController = require('../controllers/commentController');

// GET /api/comments - Get all comments
router.get('/', commentController.getAllComments);

// POST /api/comments - Create new comment
router.post('/', commentController.createComment);

// GET /api/comments/:id - Get comment by ID
router.get('/:id', commentController.getCommentById);

// PUT /api/comments/:id - Update comment
router.put('/:id', commentController.updateComment);

// DELETE /api/comments/:id - Delete comment
router.delete('/:id', commentController.deleteComment);

// POST /api/comments/:id/likes - Like comment
router.post('/:id/likes', commentController.likeComment);

// DELETE /api/comments/:id/likes - Unlike comment
router.delete('/:id/likes', commentController.unlikeComment);

module.exports = router;
```

Part 3: Testing Your API

Test your API using Postman or curl

Try sending these requests to test the API:

Get all users:

```
curl -X GET http://localhost:3000/api/users
```

Get all videos:

```
curl -X GET http://localhost:3000/api/videos
```

Get user by ID:

```
curl -X GET http://localhost:3000/api/users/1
```

Get video by ID:

```
curl -X GET http://localhost:3000/api/videos/1
```

Get user’s videos:

```
curl -X GET http://localhost:3000/api/users/1/videos
```

Get video comments:

```
curl -X GET http://localhost:3000/api/videos/1/comments
```